

RecipeVault

A Cozy Dessert Recipe Assistant Powered by a RAG Pipeline

Final Project Report

Hemamalini Kamaladhasan

Roll Number: ZDA24B025

Z2004: Database Management Systems · Track A: RAG Pipeline

IIT Madras Zanzibar

github.com/hema200528/RecipeVault

1. Introduction

RecipeVault is a cozy dessert recipe assistant built on a PostgreSQL database with a Retrieval-Augmented Generation (RAG) pipeline. A user simply asks for a dessert in plain English for example, “a chocolate cake under five steps” and the app returns matching recipes drawn directly from the database.

I chose this project because searching for recipes online is often an overwhelming experience. Most recipe websites are cluttered with long stories, advertisements, and pop-ups before you ever reach the actual recipe. I wanted to build something that does the opposite: a calm, simple, and soothing app where you type what you are craving and immediately get clean, relevant results with nothing in the way.

What makes RecipeVault unique is this focus on a gentle, easy-to-navigate experience combined with intelligent search. Instead of matching only exact keywords, it understands the meaning behind a request, and it presents the results in a warm, friendly interface designed to feel pleasant rather than busy. The goal was an app that is both technically solid underneath and comforting to use on the surface.

2. Problem Statement

Finding a suitable recipe online is harder than it should be. Search results are crowded with content that surrounds the recipe rather than the recipe itself, and ordinary keyword search is rigid. Searching for “citrus dessert” will miss a lemon tart or a key lime pie simply because those words do not appear. There is a need for a focused tool that lets a person describe what they want naturally and receive clean, relevant, trustworthy results from a reliable source of data.

RecipeVault addresses this by storing a well-structured collection of dessert recipes in a relational database and layering a semantic search over it, so that natural-language cravings are matched by meaning and answered with real recipes from the database.

3. The Track Chosen

This project follows Track A: the RAG Pipeline track. This track was chosen because it combines two areas I wanted to work on careful relational database design, and a modern retrieval layer that connects natural language to that database. It allowed me to demonstrate solid schema design, normalisation, querying, and indexing, while also building an application that uses embeddings and vector search to answer questions in a grounded way.

4. The Database Tool

The main database tool is PostgreSQL. PostgreSQL was chosen because it handles relational integrity well on primary keys, foreign keys, constraints, and joins and supports advanced features such as window functions, common table expressions, stored procedures, and detailed query analysis through EXPLAIN ANALYZE, all of which this project uses.

Alongside PostgreSQL, the application uses Pinecone, a managed cloud vector database, to store the recipe embeddings and perform semantic search. PostgreSQL remains the source of truth for all recipe data, while Pinecone handles the meaning-based search. The two work together: Pinecone decides which recipes are relevant, and PostgreSQL provides the real recipe content.

5. Dataset, Exploration, and Process

The data originates from the RecipeNLP dataset on Kaggle, which contains around 100,000 recipes. Exploring the dataset showed that it covered all kinds of food, with each row holding a title, a list of ingredients, a list of directions, and named-entity tags. Since my focus was desserts and baking, I filtered the data by title keywords down to 1,755 dessert recipes.

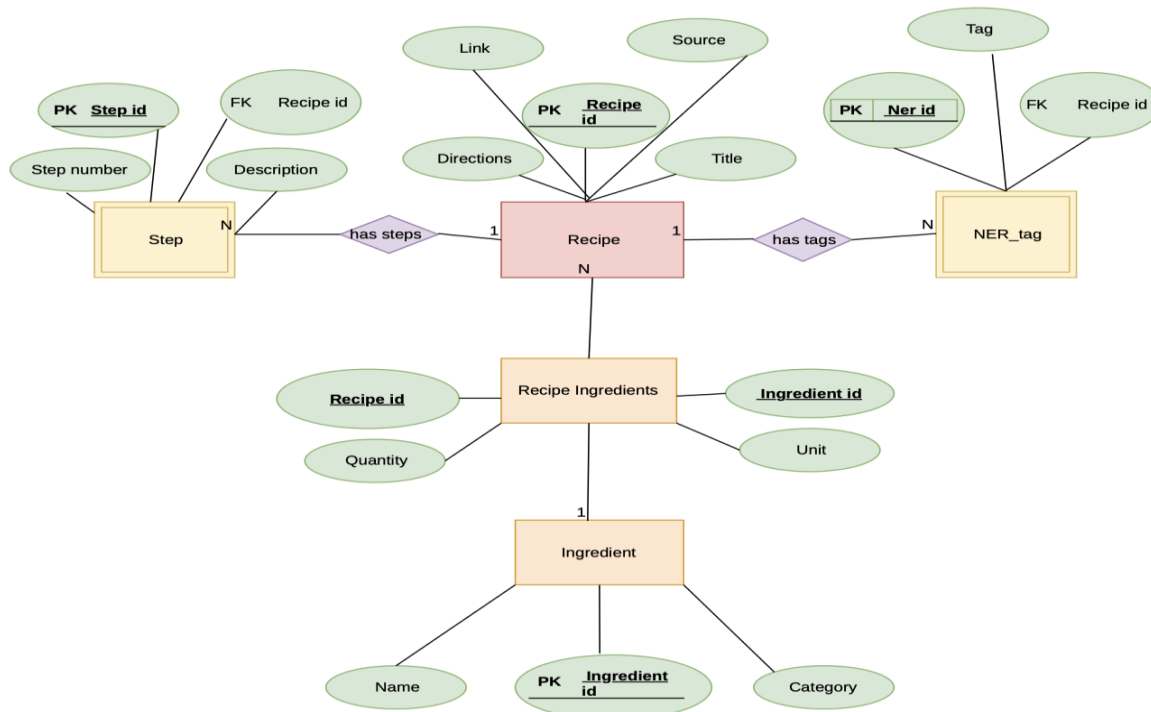
During exploration I also noticed the dataset had no notion of occasion or flavour. Because my search idea depended on these, I added two new columns, occasion and flavour_profile, and classified each recipe into them using keyword rules on its title and tags for example, a title containing “chocolate” maps to the Chocolate flavour, and one mentioning “Christmas” maps to the Christmas occasion.

The loading process used a Python script that read each recipe, parsed its ingredient and step lists out of single text fields, and distributed the data across the five tables, skipping duplicate titles. During final testing I found that the step and tag tables had duplicate rows from an earlier double-run of the import, and removed them with a self-join DELETE so that each row appears only once.

Table	Rows
Recipe	1,755
Ingredient	9,277
Recipe_Ingredient	17,015
Step	13,762
NER_Tag	15,333

6. Entity-Relationship Diagram

The ER diagram below shows the five entities and the relationships between them, drawn in Chen notation.



7. Schema and Normalisation

The database has five tables, all normalised to Third Normal Form. The design separates each entity so that no information is repeated and every non-key attribute depends only on its table's key.

Table	Primary Key	Key columns / foreign keys
Recipe	recipe_id	title, directions, occasion, flavour_profile
Ingredient	ingredient_id	name, category
Recipe_Ingredient	recipe_id + ingredient_id	FK→Recipe, FK→Ingredient, quantity, unit
Step	step_id	FK→Recipe, step_number, description
NER_Tag	ner_id	FK→Recipe, tag

7.1 Key Schema Definition

```

CREATE TABLE Recipe (
  recipe_id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL UNIQUE,

```

```

directions    TEXT NOT NULL,
occasion      VARCHAR(50),
flavour_profile VARCHAR(50)
);

CREATE TABLE Recipe_Ingredient (
  recipe_id   INT NOT NULL,
  ingredient_id INT NOT NULL,
  quantity    VARCHAR(50),
  unit        VARCHAR(50),
  PRIMARY KEY (recipe_id, ingredient_id),
  FOREIGN KEY (recipe_id) REFERENCES Recipe(recipe_id) ON DELETE CASCADE,
  FOREIGN KEY (ingredient_id) REFERENCES Ingredient(ingredient_id) ON DELETE CASCADE
);

```

7.2 Design Decisions and 3NF

- Ingredients live in their own table, so each ingredient is stored once and linked, rather than repeated as text in every recipe.
- Recipe_Ingredient is a junction table that resolves the many-to-many relationship and holds the quantity and unit, which depend on the recipe-ingredient pair.
- Step and NER_Tag are weak entities, each row belonging to exactly one recipe.
- All foreign keys use ON DELETE CASCADE to keep the data clean when a recipe is removed.
- Each table meets 3NF: every non-key attribute depends only on the whole key, with no partial or transitive dependencies.

8. Sample Queries

Ten queries were written covering aggregations, joins, subqueries, CTEs, and window functions. Two representative examples are shown below.

8.1 Aggregation - recipes per flavour

```

SELECT flavour_profile, COUNT(*) AS recipe_count
FROM recipe
GROUP BY flavour_profile
ORDER BY recipe_count DESC;

```

8.2 CTE - classify recipes by difficulty

```

WITH step_counts AS (
  SELECT r.recipe_id, r.title, COUNT(s.step_id) AS steps
  FROM recipe r LEFT JOIN step s ON r.recipe_id = s.recipe_id
  GROUP BY r.recipe_id, r.title
)
SELECT title, steps,
CASE WHEN steps <= 5 THEN 'Easy'
      WHEN steps <= 10 THEN 'Medium'
      ELSE 'Hard' END AS difficulty
FROM step_counts ORDER BY steps;

```

9. Performance and Results

Five B-Tree indexes were created on the columns the application filters and joins on most: flavour_profile and occasion on Recipe, and the recipe_id foreign keys on NER_Tag, Step, and Recipe_Ingredient. Performance

was measured using EXPLAIN ANALYZE, which runs a query and reports its real execution time, before and after adding the indexes.

Metric	Before Index	After Index
Scan type	Hash Join + Seq Scan	Nested Loop + Index Scan
Execution time	9.421 ms	0.191 ms
Improvement	baseline	~49x faster

Before indexing, the planner scanned every tag row with a Hash Join. After indexing, it found the relevant recipes through the occasion index first, then looked up only their tags through the tag index in a Nested Loop. This made the query about fifty times faster and confirmed that well-chosen indexes make a large, measurable difference.

10. Stored Procedure

A stored procedure, `get_recipes_by_flavour()`, takes a flavour as input and returns all recipes of that flavour with their occasion and step count, ordered by complexity. It is saved inside the database and can be reused by name.

```
CREATE OR REPLACE FUNCTION get_recipes_by_flavour(p_flavour VARCHAR)
RETURNS TABLE (recipe_id INT, title VARCHAR, occasion VARCHAR, step_count BIGINT)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT r.recipe_id, r.title, r.occasion, COUNT(s.step_id)
    FROM recipe r LEFT JOIN step s ON r.recipe_id = s.recipe_id
    WHERE r.flavour_profile = p_flavour
    GROUP BY r.recipe_id, r.title, r.occasion
    ORDER BY COUNT(s.step_id) DESC;
END; $$;
```

11. The Application

The application is a RAG pipeline retrieves real data before answering, so results are grounded in the database rather than invented. It works in four steps:

- The user's question is turned into an embedding, a list of 384 numbers capturing its meaning, using a sentence-transformer model in Python.
- That embedding is sent to Pinecone, which holds an embedding for every recipe and returns the closest matches by meaning.
- The matching recipe IDs are used to fetch the full recipe details from PostgreSQL, so every result shown is real, stored data.
- The results are displayed with a match score and a link to the source recipe.

If Pinecone or the network is unavailable, the app automatically falls back to a PostgreSQL keyword search, so it always returns results. API keys are kept in a `.env` file that is never committed to version control.

11.1 Key Code - the retrieval function

```
def search_rag(question, top_k=5):
    query_vector = model.encode(question).tolist() # embed the question
    results = index.query(vector=query_vector,      # search Pinecone
```

```

        top_k=top_k, include_metadata=True)
    recipes = []
    for match in results['matches']:
        row = fetch_details(int(match['id']))    # fetch from PostgreSQL
        recipes.append(row)
    return recipes

```

12. Final Design of the App

The finished application has two interfaces sharing the same pipeline. The command-line version prints matching recipes for a typed question. The web version presents the same results in a warm, dessert-cafe theme designed to feel soothing rather than cluttered the opposite of a typical busy recipe website.

The web interface has a clean search box, tappable flavour filters, and gentle animations. Each result appears as a card showing the recipe’s flavour, occasion, and step count, with a match score. A “view recipe” button opens a pop-up with the full list of ingredients and the ordered cooking steps for that recipe, fetched live from the database. The overall design keeps the experience simple and calm: type a craving, get clean results, open the one you like.

13. Fallback Plan

A key part of the design is that the application keeps working even if the Pinecone vector service or the internet connection is unavailable. When the app starts, it tests the connection to Pinecone. If it cannot reach it, the app prints “Pinecone unavailable using database fallback” and automatically switches to a PostgreSQL keyword search on recipe titles and flavours, using a SQL LIKE match. The app never crashes; it always returns real recipes from the database.

The difference is that the normal RAG mode matches by meaning, while the fallback matches by keyword. The table below shows sample inputs and the expected outputs in each mode, so the behaviour can be verified even with the network down.

Sample input	Normal output (RAG mode)	Fallback output (Pinecone down)
chocolate	Chocolate recipes ranked by semantic match score	Recipes matched by LIKE '%chocolate%' on title / flavour
lemon	Citrus desserts found by meaning (lemon, lime, etc.)	Recipes containing “lemon” in the title or flavour
cake	Cakes ranked by semantic match	Recipes containing “cake” in the title

In both modes the user receives a clean list of real dessert recipes from the database, so a live demonstration is safe even without connectivity.

14. Challenges Faced

Building RecipeVault involved several real challenges, each of which taught me something about working with data and databases.

- Loading the data was the hardest part. The ingredients and steps were stored as text lists packed inside single CSV cells, so I had to parse them in Python and split them into separate rows across the Ingredient, Recipe_Ingredient, and Step tables to fit my normalised schema.

- Some ingredient names came through with quantities still attached, such as “2 cups flour” instead of just “flour”, because of how the source data was written. I noted this as a limitation to fix in future work.
- During final testing I discovered that my steps and tags had been duplicated because the import script had been run twice. I caught this, backed up the database, and wrote a self-join DELETE that keeps one clean copy of each row.
- I had two PostgreSQL servers installed on my machine that competed for the same port, which temporarily caused the app to lose connection to its tables. I diagnosed the conflict and made sure the correct server was running.
- Connecting the web interface to the live RAG pipeline took careful debugging. I had to set up a small Flask backend so the cute web page could call the same embedding, vector search, and database logic as the terminal app.

15. Future Work

- Deploy RecipeVault as a live website so anyone can use it without running it locally.
- Expand the dataset well beyond the current 1,755 recipes for greater variety.
- Add more cuisines beyond desserts, so the app becomes a general recipe assistant.
- Clean the ingredient names by separating quantities from the ingredient itself, enabling precise ingredient-based search.
- Add user features such as saving favourite recipes and creating personal collections.

16. Conclusion

RecipeVault began with a simple frustration that finding a recipe online is needlessly overwhelming and answered it with a calm, focused app backed by a carefully designed database. It combines a normalised five-table schema, a real dessert dataset, a full set of SQL queries, a measurable 49x performance gain from indexing, and a working RAG pipeline that answers natural-language cravings with grounded results. Every design decision was guided by two goals: a technically sound database underneath, and a soothing, easy experience on top.

AI Usage Disclosure

Claude (Anthropic) was used for syntax suggestions, debugging help, and structuring this report and the application code. All design decisions, data choices, query logic, and execution were carried out and verified independently, and the system was built and tested on my own machine.